



LOST BOYS CYBER

THREAT INTELLIGENCE • MALWARE ANALYSIS • DETECTION ENGINEERING



From SQLi to RCE – Exploiting LangGraph’s Checkpointer

Vulnerability Report

Classification	TLP:CLEAR
Published	2026-06-12
Version	1.0
Author	Lost Boys Cyber
Report Type	Vulnerability Report

TLP:CLEAR | Lost Boys Cyber | From SQLi to RCE – Exploiting LangGraph’s Checkpointer - Vulnerability Report

Published: 2026-06-12 | TLP:CLEAR | Version: 1.0 | Author: Lost Boys Cyber

AI Generation: This report was generated with AI assistance and is provided for informational purposes only. All findings, IOCs, detection rules, hunting queries, attribution assessments, and recommendations must be independently reviewed and validated by a qualified security professional before operational use.

Table of Contents

1. Executive Summary
2. Intelligence Requirement
3. Source Review and Confidence
4. Key Findings
5. Threat Context
6. Detection and Hunting
7. Recommendations
8. References

1. Executive Summary

Check Point Research published a technically credible chain showing how insecure persistence handling in self-hosted LangGraph deployments can move from query manipulation to server-side code execution. The most serious path combines `CVE-2025-67644`, a SQL injection in the SQLite checkpointer's metadata filter handling, with `CVE-2026-28277`, an unsafe msgpack deserialization condition in checkpoint loading. In parallel, `CVE-2026-27022` introduces a related injection class in the Redis checkpointer. The common risk theme is not “all LangGraph is compromised,” but rather that stateful agent deployments become dangerous when applications expose checkpoint history filtering to untrusted input and rely on persistence-layer components that interpolate attacker-controlled data or deserialize attacker-controlled checkpoint bytes.

The exposure is concentrated in organizations self-hosting LangGraph with SQLite or Redis-backed checkpointing, especially where application code exposes `get\_state\_history()` or equivalent history-listing logic to user-controlled `filter` input. Check Point explicitly states LangChain's managed cloud service, LangSmith Deployment (formerly LangGraph Platform), uses PostgreSQL and is not vulnerable to the specific checkpointer issues discussed in the article. That distinction matters operationally: defenders should prioritize internally built agent services, research sandboxes, copilots, and workflow-automation APIs that persist agent state locally or in Redis rather than assuming every LangChain-related workload is equally exposed.

The SQLite attack chain is the highest-priority scenario. If an attacker can control metadata filter keys, they may inject SQL into the history query and use a crafted `UNION SELECT` row to return attacker-controlled checkpoint bytes. When the vulnerable loader interprets that row as a msgpack checkpoint, unsafe object reconstruction can trigger arbitrary command execution in the application runtime. The Redis issue is serious but different in practical effect: it enables RediSearch query manipulation and cross-thread data exposure, rather than the full SQLi-to-deserialization chain described for SQLite.

Immediate defender priorities are to upgrade `langgraph-checkpoint-sqlite` to `3.0.1+`, `langgraph` to `1.0.10+`, and `@langchain/langgraph-checkpoint-redis` to `1.0.2+`; inventory all self-hosted agent deployments that persist checkpoints; identify any API surface that passes user-controlled `filter` keys into checkpoint history functions; and monitor for suspicious outbound activity, shell execution, or checkpoint-store tampering on vulnerable hosts. No stable public malware IOCs or exploitation-at-scale reporting were identified in the reviewed source set as of 2026-06-11, so behavior-based detection and rapid version governance are more valuable than signature-only controls.

Executive takeaway	Assessment
Business impact	Self-hosted LangGraph agents can expose sensitive state, break thread isolation, or reach server-side code execution if vulnerable checkpointers are reachable through attacker-controlled history filters or compromised persistence stores.
Highest-risk deployment pattern	LangGraph applications using SQLite checkpointing and exposing `get_state_history()` or equivalent list functionality to user-controlled `filter` keys.
Public exploitation status	Reviewed public sources described credible exploit mechanics but did not confirm in-the-wild exploitation or KEV inclusion as of 2026-06-11.
Immediate priority	Patch the three affected packages, remove untrusted control over checkpoint filter keys, and investigate checkpoint-store access paths in self-hosted agent services.

2. Intelligence Requirement

This review addresses the following intelligence requirement: what did the 2026 Check Point Research publication actually prove about LangGraph's persistence layer, which deployments are materially exposed, how should defenders separate the SQLite RCE chain from the Redis query-injection issue, and what practical response actions should engineering, AppSec, and detection teams take next?

Question	Assessment
What vulnerabilities are covered?	`CVE-2025-67644` (SQLite metadata-filter SQL injection), `CVE-2026-28277` (unsafe msgpack checkpoint deserialization), and `CVE-2026-27022` (Redis checkpointer query injection).
What is the most dangerous chain?	SQL injection in the SQLite checkpointer can inject attacker-controlled checkpoint rows that are later deserialized, creating a path to arbitrary code execution.
What must be true for exploitation?	The application must self-host LangGraph, use the affected checkpoint implementation, and expose attacker influence over checkpoint history filtering or persisted checkpoint bytes.
Is LangSmith Deployment affected?	Reviewed source material says no for this issue set because the managed service uses PostgreSQL rather than the vulnerable SQLite or Redis checkpointers.
Are all LangGraph users equally exposed?	No. Exposure is concentrated in self-hosted deployments with SQLite or Redis checkpointing and unsafe API design around `filter` handling or checkpoint persistence.
What matters most to defenders?	Fast package upgrades, removal of user control over checkpoint filter keys, store-hardening, and telemetry that correlates checkpoint access with suspicious process or network behavior.

3. Source Review and Confidence

Source	Contribution	Confidence
Check Point Research article (2026-06-11)	Primary exploit narrative, attack chain explanation, package versions, preconditions, and disclosure timeline.	High
GitHub Advisory `GHSA-9rwj-6rc7-p77c` / `CVE-2025-67644`	Confirms the SQLite metadata-filter SQL injection, affected versions `<3.0.1`, and patched version `3.0.1`.	High
GitHub Advisory `GHSA-g48c-2wqr-h844` / `CVE-2026-28277`	Confirms the unsafe msgpack deserialization issue in `langgraph` `<= 1.0.9`, patched in `1.0.10`, and clarifies this is primarily a post-exploitation / defense-in-depth issue unless checkpoint bytes are attacker-controlled.	High
GitHub Advisory `GHSA-5mx2-w598-339m` / `CVE-2026-27022`	Confirms the Redis checkpointer query-injection issue, attack surface, and patched version `1.0.2`.	High
NVD record for `CVE-2025-67644`	Independent public cataloging of the SQLite SQL injection issue and severity metadata used by downstream scanners.	High
NVD/OpenCVE secondary cataloging for `CVE-2026-28277` and `CVE-2026-27022`	Supports public dissemination, severity context, and the absence of reviewed exploitation reporting.	Medium

Confidence is high that the three vulnerabilities are real, patched, and materially important for self-hosted LangGraph environments because the reviewed source set includes both the Check Point technical article and upstream GitHub security advisories for each issue. Confidence is also high on the specific fixed versions for `langgraph-checkpoint-sqlite` and `@langchain/langgraph-checkpoint-redis`, and high that `langgraph` `1.0.10` addresses the unsafe msgpack-deserialization issue because that version boundary is stated in the upstream advisory.

Confidence is medium on the exact universality of the SQLi-to-RCE path because the exploit chain depends on deployment-specific preconditions: the application must expose a vulnerable checkpoint history function to user-controlled filter keys and must deserialize attacker-controlled checkpoint content. The article demonstrates plausibility and code-level mechanics, but reviewed public sources do not establish that every self-hosted LangGraph service naturally exposes those conditions by default.

A noteworthy source discrepancy is that Check Point's disclosure timeline references `langgraph-checkpoint 4.0.1` for the deserialization fix, while the upstream GitHub advisory for `CVE-2026-28277` states the affected package is `langgraph` `<= 1.0.9` and the patched version is `1.0.10`. This report treats the upstream advisory as authoritative for package/version remediation and treats the article's timeline wording as likely imprecise package naming rather than a stronger remediation source.

4.1 Core Vulnerability Metadata

Field	Value
Report focus	LangGraph persistence-layer vulnerability cluster with one primary SQLite RCE chain and one parallel Redis query-injection issue
Primary CVEs	`CVE-2025-67644`, `CVE-2026-28277`, `CVE-2026-27022`
Primary affected ecosystem	Self-hosted LangGraph / LangChain agent deployments using persistence backends
SQLite SQLi package	`langgraph-checkpoint-sqlite`
SQLite SQLi affected versions	`< 3.0.1`
SQLite SQLi fixed version	`3.0.1`
Msgpack deserialization package	`langgraph`
Msgpack deserialization affected versions	`<= 1.0.9`
Msgpack deserialization fixed version	`1.0.10`
Redis query injection package	`@langchain/langgraph-checkpoint-redis`
Redis query injection affected versions	`< 1.0.2`
Redis query injection fixed version	`1.0.2`
Managed cloud exposure	Check Point states LangSmith Deployment uses PostgreSQL and is not vulnerable to the discussed SQLite / Redis issues
Reviewed public exploitation status	No reviewed source confirmed in-the-wild exploitation as of 2026-06-11

4.2 How the SQLite Attack Chain Works

Step	Mechanism	Why it matters
1	The application accepts attacker-influenced	This creates the entry point into the

	`filter` keys for checkpoint history listing.	vulnerable query builder.
2	The SQLite checkpointer interpolates the filter key into `json_extract(... '\$.{query_key}')` without safe validation.	This enables SQL injection via the metadata key rather than only the value.
3	The attacker injects a `UNION SELECT` row into the query result set.	This lets the attacker return fake checkpoint data without needing a legitimate stored checkpoint.
4	The returned row includes attacker-controlled bytes in the `checkpoint` field and a `type` value of `msgpack`.	The injected row is positioned to reach the loader's deserialization path.
5	Vulnerable checkpoint loading reconstructs the msgpack payload through a custom extension hook.	Unsafe object reconstruction can invoke attacker-controlled Python functions such as `os.system()`.
6	The application runtime executes the supplied command.	The chain results in server-side code execution under the agent service's privileges.

4.3 Exposure Patterns and Preconditions

Exposure pattern	Why it matters	Confidence
Self-hosted LangGraph services exposing `get_state_history()` or equivalent history-listing APIs to untrusted clients	This is the precondition explicitly described by Check Point for the SQLite and Redis injection issues.	High
Internal agent services persisting checkpoints in SQLite on the same application host	Successful exploitation can turn a query flaw into runtime command execution on the service host.	High
Redis-backed deployments relying on thread isolation for multi-tenant agent state	Query injection can manipulate RediSearch filters and expose data outside intended thread boundaries.	High
Managed LangSmith Deployment using PostgreSQL	Reviewed public sources say this specific issue set does not apply.	High
Deployments that never expose user-controlled filter keys and tightly control checkpoint-store writes	The SQLi and deserialization preconditions may not be reachable even if a vulnerable package version is present.	Medium

4.4 Analyst Findings

Finding	Defensive implication
The highest-risk issue is not generic “LangGraph RCE,” but a conditional chain dependent on unsafe API exposure and vulnerable SQLite persistence.	Triage should focus on reachable application paths, not just package presence.
`CVE-2026-28277` is best understood as a blast-radius escalator after checkpoint-store compromise or injected checkpoint retrieval.	Defenders should harden storage access even after patching the direct SQLi entry point.
The Redis issue is serious for confidentiality and tenant isolation even without the exact same RCE path.	Multi-tenant agent deployments using Redis should be reviewed for cross-thread data exposure and query manipulation risk.

Public reporting did not provide stable victim-side IOCs such as attacker domains, hashes, or exploit kit artifacts.	Detection engineering should emphasize version inventory, API misuse, suspicious query behavior, and post-exploitation telemetry.
--	---

5. Threat Context

This vulnerability cluster is a strong example of how AI-agent infrastructure inherits classic application-security problems while amplifying the blast radius through persistence and tool execution. LangGraph's value proposition is durable state across steps, threads, and agents. That same persistence layer becomes a high-value attack surface when developers expose history or filtering operations to clients without a strict trust boundary. In a conventional CRUD service, metadata-filter injection may “only” disclose or tamper with database rows. In a stateful agent platform, injected rows can influence what the runtime believes is legitimate serialized execution state.

The SQLite chain is especially relevant to organizations rapidly piloting internal AI tools, copilots, and autonomous workflows. These services are often self-hosted, built quickly, and exposed behind lightweight internal APIs where developers may trust application-layer filtering too much. If such a service also has filesystem access, cloud credentials, developer secrets, or downstream tool integrations, a successful exploit can move beyond the checkpoint database into broader application compromise.

The Redis issue should not be minimized simply because the article's marquee narrative focuses on SQLite-to-RCE. In real multi-user agent deployments, cross-thread data exposure or access-control bypass can itself be severe: agent state may include prompts, secrets, tool outputs, internal planning context, customer data, or session-specific credentials. For some organizations, that confidentiality impact is operationally significant even without direct command execution.

MITRE ATT&CK mapping	Relevance	Confidence
`T1190` Exploit Public-Facing Application	Plausible when a self-hosted LangGraph API exposes checkpoint history filtering to remote clients and the attacker exploits the vulnerable persistence path through that interface.	Medium
`T1059.004` Command and Scripting Interpreter: Unix Shell	Relevant to the SQLite chain because Check Point's demonstrated unsafe object reconstruction can invoke <code>`os.system()`</code> with attacker-controlled commands.	Medium
`T1210` Exploitation of Remote Services	Plausible where the vulnerable agent service is reachable over an internal or internet-facing API and exploitation traverses that service boundary.	Low
`T1530` Data from Cloud Storage Object	Scenario-dependent follow-on effect if compromised agent services can reach cloud-backed stores or internal data through attached credentials and tools.	Low
Additional ATT&CK techniques	Not assigned with high confidence because the reviewed public sources do not document observed intrusion operations beyond the exploit mechanics themselves.	High

6. Detection and Hunting

Detection should assume sparse static IOCs and focus on four layers: vulnerable-version discovery, API-path review, checkpoint-store integrity, and post-exploitation behavior. The most useful first step is to inventory every environment using LangGraph persistence, then determine whether user-controlled input can reach checkpoint history functions. After that,

defenders should watch for suspicious query construction, unexpected checkpoint rows, unusual shell execution by the agent runtime, and outbound connections immediately following checkpoint-load events.

6.1 Detection Logic Summary

Signal	Telemetry source	Analytic goal
Hosts or containers running <code>`langgraph-checkpoint-sqlite < 3.0.1`, `langgraph <= 1.0.9`, or `@langchain/langgraph-checkpoint-redis < 1.0.2`</code>	SBOMs, package manifests, CI dependency scans, container inventories	Identify vulnerable deployments before exploit telemetry appears
Requests that hit checkpoint history endpoints with unusual <code>`filter`</code> key names containing quotes, operators, braces, or query syntax	API gateway logs, application logs, WAF telemetry	Surface probable injection attempts against the list/history function
Checkpoint history queries returning unexpected row shapes or <code>`type=msgpack`</code> entries from unusual threads	Application debug logs, database auditing, custom telemetry around checkpoint loads	Identify fake-row injection or suspicious checkpoint manipulation
Agent-service processes spawning shells or unexpected child processes after checkpoint retrieval	EDR process telemetry, Sysmon, auditd, container runtime logs	Detect possible command execution caused by unsafe deserialization
Redis query anomalies or cross-thread result patterns in multi-tenant deployments	Redis/RediSearch logs, app telemetry, tenant access reviews	Detect thread-boundary bypass and unexpected checkpoint disclosure

Analyst caveat: many legitimate agent workflows will naturally query history and deserialize checkpoints. Triage should prioritize novelty, malformed filter keys, tenants crossing thread boundaries, checkpoint-store writes from unexpected components, and process activity that deviates from the agent service's normal behavior.

6.2 Example Sigma Logic

```

title: Suspicious LangGraph Checkpoint History Filter Injection Attempt
id: cve-2025-67644-langgraph-filter-key-injection
status: experimental
description: Detects likely malicious filter-key input sent to LangGraph checkpoint history APIs,
especially where get_state_history or similar list methods are exposed to untrusted callers.
logsource:
  product: web-application
  category: application
  service: langgraph
detection:
  selection_endpoint:
    RequestPath|contains:
      - '/state-history'
      - '/get_state_history'
      - '/checkpoints'
  selection_payload:
    RequestBody|re: '(?i)filter.*(\'|\"|\\)|\{\}|union|select|json_extract|@thread_id|\\|)'
  condition: selection_endpoint and selection_payload
falsepositives:
  - developer testing of checkpoint filter functionality
  - security validation or red-team activity against internal agent services
level: high

```

6.3 Example Hunt Queries

Microsoft Defender XDR / Sentinel-style hunt for suspicious process execution from likely LangGraph runtimes after Python-based checkpoint activity:

```
let suspiciousParents = dynamic(["python", "python3", "uvicorn", "gunicorn", "fastapi", "langgraph"]);
```



```
DeviceProcessEvents
| where Timestamp > ago(30d)
| where InitiatingProcessFileName in~ (suspiciousParents)
| where FileName in~ ("sh", "bash", "zsh", "dash", "cmd.exe", "powershell.exe")
| project Timestamp, DeviceName, AccountName, InitiatingProcessFileName, InitiatingProcessCommandLine,
FileName, ProcessCommandLine, SHA1
| order by Timestamp desc
```

Hunt for suspicious checkpoint history requests containing likely injection syntax in reverse-proxy or application logs normalized into a table named `AppRequests`:

```
AppRequests
| where Timestamp > ago(30d)
| where RequestPath has_any ("state-history", "get_state_history", "checkpoints")
| where RequestBody matches regex @"(?i)filter.*('|\\"|\\)|\\{|\\}|union|select|json_extract|@thread_id|\\|)"
| project Timestamp, Host, User, SourceIP, RequestPath, RequestBody, HttpStatus
| order by Timestamp desc
```

6.4 Hunting Guidance

Hunt question	Why it matters
Which self-hosted agent services still use SQLite or Redis checkpointing, and which package versions are deployed?	This defines the immediate remediation and validation scope.
Can any external or untrusted internal user influence `filter` keys passed into checkpoint history operations?	That is the critical reachability question for the injection issues.
Do checkpoint stores contain unexpected rows, unusual `msgpack` content, or writes from components that should only read history?	These are strong indicators of tampering or exploit staging.
Did agent-service hosts spawn shells or unusual subprocesses shortly after checkpoint retrieval or history-listing requests?	This is the most actionable behavior-based clue for attempted SQLite-chain exploitation.
Do Redis-backed multi-tenant services show evidence of thread-boundary bypass or cross-tenant checkpoint visibility?	That would indicate practical impact from `CVE-2026-27022` even without command execution.

7. Recommendations

Priority	Owner	Action
Critical	Engineering / Platform	Upgrade `langgraph-checkpoint-sqlite` to `3.0.1+`, `langgraph` to `1.0.10+`, and `@langchain/langgraph-checkpoint-redis` to `1.0.2+`, then rebuild and redeploy all affected services.
Critical	Application Security	Remove direct user control over checkpoint `filter` keys and validate that history-listing APIs enforce allowlisted fields rather than free-form metadata selectors.
Critical	Service Owners	Identify every self-hosted LangGraph deployment using SQLite or Redis persistence and document whether `get_state_history()` or similar logic is externally reachable.
High	Detection Engineering	Add application-log detections for malformed checkpoint filter input and EDR

		correlations for shell execution by Python-based agent services.
High	Infrastructure / IAM	Review what secrets, filesystem paths, cloud roles, or internal tools are reachable from agent-service runtimes so potential post-exploitation blast radius is understood.
High	Database / Platform	Restrict write access to checkpoint stores, enable auditing where feasible, and separate read/write trust zones for persistence backends used by agent services.
Medium	QA / Secure Development	Add regression tests that attempt malicious filter keys and malicious serialized checkpoints so future persistence-layer regressions fail closed.
Medium	Threat Intelligence	Monitor LangGraph, LangChain, GitHub advisories, and public exploit reporting for any update to exploitation status, package naming clarifications, or additional affected components.

Additional analyst guidance:

- Treat this as an architecture review issue as well as a patching issue. The report's strongest lesson is that agent-state persistence should be modeled as part of the application attack surface, not as an inert storage detail.
- Prioritize services that combine checkpoint persistence with tool execution, cloud credentials, code-repository access, or developer secrets, because these environments offer the highest-value post-exploitation outcomes.
- No reliable public IOC set was identified in the reviewed sources. Do not overpromise indicator-based detection where version governance and behavior telemetry will perform better.

8. References

- 1. Check Point Research, "From SQLi to RCE - Exploiting LangGraph's Checkpointer" (2026-06-11): <https://research.checkpoint.com/2026/from-sqli-to-rce-exploiting-langgraphs-checkpointer/>
- 2. GitHub Advisory `GHSA-9rwj-6rc7-p77c` / `CVE-2025-67644`: <https://github.com/advisories/GHSA-9rwj-6rc7-p77c>
- 3. GitHub Advisory `GHSA-g48c-2wqr-h844` / `CVE-2026-28277`: <https://github.com/advisories/GHSA-g48c-2wqr-h844>
- 4. GitHub Advisory `GHSA-5mx2-w598-339m` / `CVE-2026-27022`: <https://github.com/advisories/GHSA-5mx2-w598-339m>
- 5. NVD record for `CVE-2025-67644`: <https://nvd.nist.gov/vuln/detail/CVE-2025-67644>
- 6. NVD record for `CVE-2026-28277`: <https://nvd.nist.gov/vuln/detail/CVE-2026-28277>
- 7. NVD record for `CVE-2026-27022`: <https://nvd.nist.gov/vuln/detail/CVE-2026-27022>